

ATIVIDADE: ESTENDENDO A CALCULADORA (2)

1. Estenda as funcionalidades da calculadora incluindo operações como raiz quadrada e potenciação.

```
public double sqrt(double a) {  
    if (a < 0) {  
        throw new IllegalArgumentException("Não existe raiz quadrada real de número negativo.");  
    }  
    return Math.sqrt(a);  
}  
  
public double power(double base, double exponent) {  
    return Math.pow(base, exponent);  
}
```

2. Considere cenários que envolvam:
 - Números inteiros e decimais;
 - Valores limites (mínimo e máximo);
 - Divisão por zero (lançamento de exceção);
 - Operações com números negativos.

```
@Test  
@DisplayName("Teste de Raiz Quadrada com números positivos e decimais")  
void testSqrt() {  
    assertEquals(expected: 5.0, calculator.sqrt(a: 25), message: "Raiz de 25 deve ser 5");  
    assertEquals(expected: 1.5, calculator.sqrt(a: 2.25), message: "Raiz de 2.25 deve ser 1.5");  
}  
  
@Test  
@DisplayName("Teste de Raiz Quadrada de zero")  
void testSqrtZero() {  
    assertEquals(expected: 0.0, calculator.sqrt(a: 0), message: "Raiz de 0 deve ser 0");  
}  
  
@Test  
@DisplayName("Validação de exceção para Raiz de número negativo")  
void testSqrtNegative() {  
    assertThrows(IllegalArgumentException.class, () -> calculator.sqrt(a: -9),  
        message: "Deve lançar IllegalArgumentException para raizes negativas");  
}
```

3. Considere a utilização das diferentes alternativas de teste indicadas anteriormente.

```

@ParameterizedTest
@CsvSource({
    "2, 3, 8",
    "10, -1, 0.1",
    "2.5, 2, 6.25",
    "4, 0.5, 2"
})
@DisplayName("Testes Parametrizados de Potenciação")
void testPower(double base, double exp, double expected) {
    assertEquals(expected, calculator.power(base, exp), delta: 0.0001);
}

@Test
@DisplayName("Teste de limites e valores máximos/mínimos")
void testBoundaries() {
    double result = calculator.multiply(Double.MAX_VALUE, b: 2);
    assertTrue(Double.isInfinite(result), message: "Multiplicação além do limite deve resultar em Infinito");

    double small = calculator.divide(Double.MIN_VALUE, b: 2.0);
    assertEquals(expected: 0.0, small, delta: 0.0);
}

```

```

@Test
@DisplayName("Operações com números negativos (Subtração e Multiplicação)")
void testNegativeOperations() {
    assertEquals(expected: -15.0, calculator.add(a: -10, b: -5));
    assertEquals(expected: 50.0, calculator.multiply(a: -10, b: -5));
    assertEquals(expected: -2.0, calculator.divide(a: 10, b: -5));
}

```